ELO320 - Estructuras de Datos y Algoritmos
Introducción a C
Memoria Dinámica

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

1er Semestre 2025

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Clase Activa: Módulo 1

Ejercicio 1.1

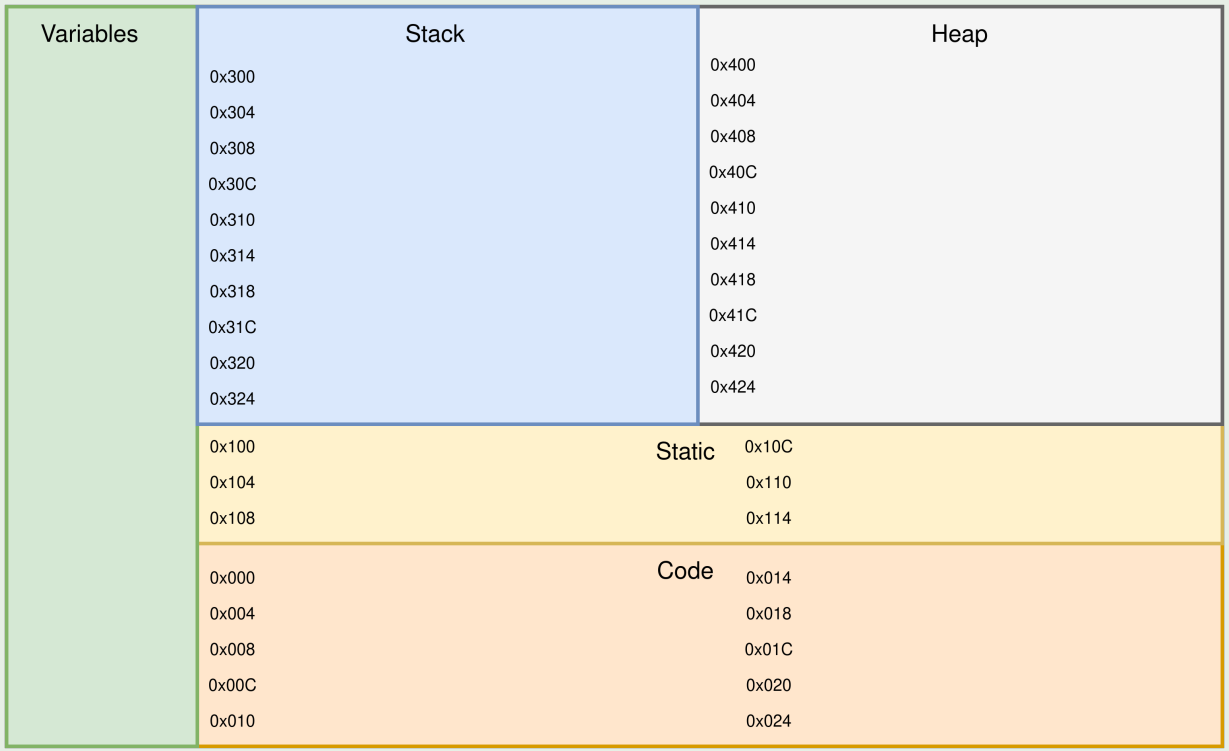
Cree un programa que permita trabajar con un entero sin utilizar una **variables de stack o static** para su *uso directo*: asignar, almacenar y utilizar.

Aprendemos: uso de memoria heap; reserva y liberación de memoria.

Clase Activa: Módulo 1

Ejercicio 1.2

Dibuje la asignación de memoria en del Ejercicio 1.1. Puede utilizar el siguiente esquema, si estima conveniente:



Aprendemos: uso de memoria heap; reserva y liberación de memoria.

Clase Activa: Módulo 2

Ejercicio 2.1

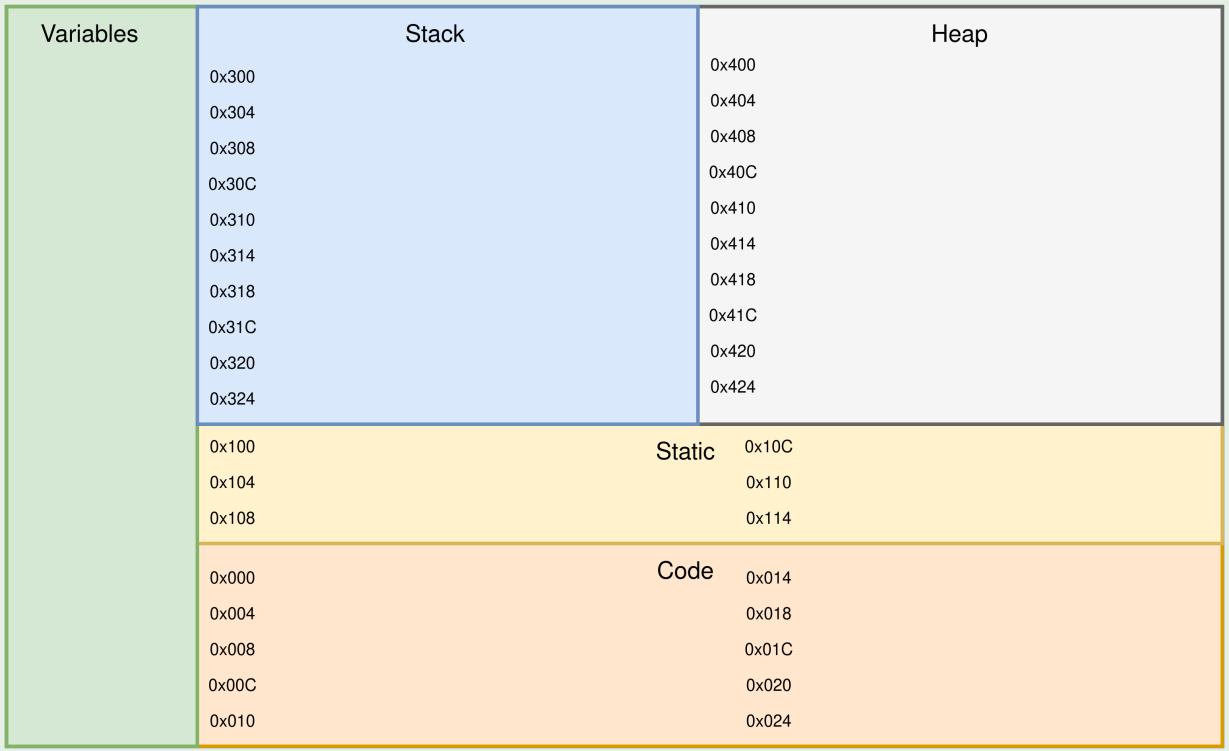
Cree un programa que permita insertar un string letra por letra, utilizando la cantidad exacta de memoria cada vez que se ingresa una letra. No puede crear arreglos fijos.

Aprendemos: uso de memoria heap; reserva, reasignación y liberación de memoria.

Clase Activa: Módulo 2

Ejercicio 2.2

Dibuje la asignación de memoria en del Ejercicio 2.1. Puede utilizar el siguiente esquema, si estima conveniente:



Aprendemos: uso de memoria heap; reserva, reasignación y liberación de memoria.

Clase Activa: Módulo 3

Ejercicio 3.1

Cree una función que permita crear un valor de un tipo de dato de su elección en **memoria heap**. Fuera de esta función, por ejemplo, en la función main, este valor debe ser utilizado. Debe liberar este elemento en la función main().

Aprendemos: retorno de punteros.

Clase Activa: Módulo 4

Analice la siguiente función, describiendo que realiza, y que problemas tiene. Luego, repárela.

Ejercicio 4.1

```
int * init_array(){

        int * a = NULL;
        int cant = 1;
        while(cant <= 10){
                a=malloc(sizeof(int) * cant);
                a[cant-1]=2*cant;
                cant++;
        }

        return a;
}
```

Clase Activa: Módulo 4

Analice la siguiente función, describiendo que realiza, y que problemas tiene. Luego, repárela.

Ejercicio 4.2

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct pukamon{
        char * nom;
        float altura;
        float peso;
}pkmn;

int main () {
        pkmn * P1 = malloc(sizeof(*P1));
        pkmn * P2 = malloc(sizeof(*P2));
        strcpy(P1->nom,"Pukachu"); P1->altura = 40.6; P1->peso = 6.0;
        strcpy(P2->nom , "Eevy");  P2->altura = 30.5; P2->peso = 6.5;
        printf("%s: %.2f %.2f \nvs \n %s: %.2f %.2f\n",
                P1->nom, P1->altura , P1->peso ,
                P2->nom, P2->altura , P2->peso);
        free(P1);
        free(P2);
        return 0;
}
```

Información de Vital Importancia para la Clase Activa[1].

[1] Depende exclusivamente de uds. leerla y comprenderla.

Memoria dinámica.

La memoria dinámica es aquella que cae bajo control del programador y se almacena el **heap** de la memoria.
Es un elemento necesario porque:

- Los arreglos fijos son espacios de memoria contigua cuyo tamaño no es modificable.
- ¿Qué pasa si definimos un tamaño y es insuficiente?
- ¿Qué pasa si el arreglo reserva más memoria que su uso real?

Memoria dinámica.

- Memoria Dinámica: Se reserva memoria en el heap.
- Se accede a esta memoria mediante punteros.
- 3 instrucciones básicas de `stdlib.h`:
  - `malloc()`: reserva de memoria.
  - `realloc()`: reasignación de memoria.
  - `free()`: liberación de memoria.
- `valgrind`: software para debugging de temas relacionados con memoria.

Memoria dinámica: malloc

- Instrucción para la reserva de memoria.
- Entrega como resultado la dirección de memoria donde inicia la reserva.
- **Necesitamos punteros**.
- Operador `sizeof` como base del tamaño a reservar.

C

```
#include<stdio.h>
#include<stdlib.h>

int main(int argc, char **argv){
        int *arreglo=NULL;

        arreglo = malloc(10 * sizeof(*arreglo));
//      arreglo = malloc(10 * sizeof(int));
//      arreglo = malloc(10 * sizeof *arreglo);

        // sizeof es un operador: sizeof(tipo_de_dato) o sizeof expresion
        // cast no es necesario, compilador hace coerción.
        // solo para compatibilidad con C++, se usa
//      arreglo = (int *) malloc(10 * sizeof(int));
        return 0;
}
```

Memoria dinámica: `realloc`

- Instrucción para la reasignar memoria ya reservada.
- Si se agranda la memoria, esta puede cambiar completamente de dirección.

C

```c
#include<stdio.h>
#include<stdlib.h>

int main(int argc, char **argv){
        int *arreglo=NULL;

        arreglo = malloc(10 * sizeof(*arreglo));

        // libera 5 ints, mantiene dir.
        arreglo = realloc(arreglo, 5 * sizeof(int));

        // se expande en 15 ints, mantiene dir si existe
        // espacio contiguo, sino se cambia.
        arreglo= realloc(arreglo, 20 * sizeof * arreglo);

        return 0;
}
```

Memoria dinámica: free

- Instrucción para liberar la memoria reservada en heap.
- Los ejemplos anteriores no liberaban memoria.
- Sirve para liberar los recursos ociosos y no acaparar recursos en el manejo (memory leaks).

C

```
#include<stdio.h>
#include<stdlib.h>

int main(int argc, char **argv){
        int *arreglo=NULL;

        arreglo = malloc(10 * sizeof(*arreglo));

        // libera 5 ints, mantiene dir.
        arreglo = realloc(arreglo, 5 * sizeof(int));

        // se expande en 15 ints, mantiene dir si existe
        // espacio contiguo, sino se cambia.
        arreglo= realloc(arreglo, 20 * sizeof * arreglo);

        free(arreglo);
        return 0;
}
```

Fuga de memoria - Memory Leak

- **Fuga de memoria (memory leak)**.
- Pérdida de acceso a objeto de memoria reservado en el heap, por no existir variables que apunten a él.
- No se puede liberar hasta el término del programa.
- Puede saturar la memoria.

C

```
int main(int argc, char **argv){
        int *p;

        p= malloc(sizeof(int));
        p= malloc(sizeof(int));
        /* se pierde la direccion asignada anteriormente */

        free(p);

        return 0;
```

Dangling

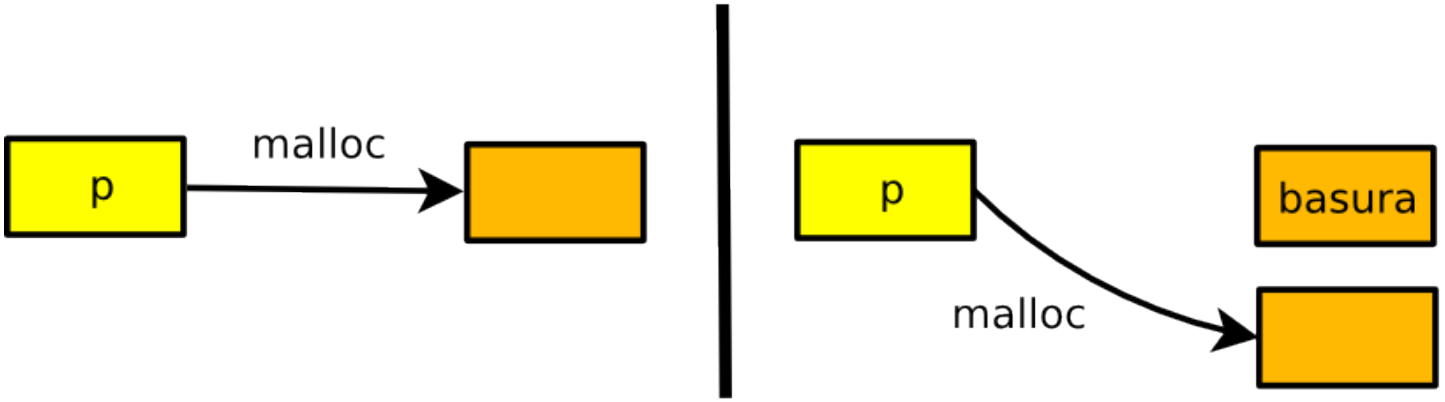
- **Dejar Colgado (dangling).**
- Puntero hacia localización de memoria del heap que ha sido liberada (e incluso nuevamente asignada).
- Genera errores poco predecibles (e.g. Segmentation fault, corrupción de memoria).

C

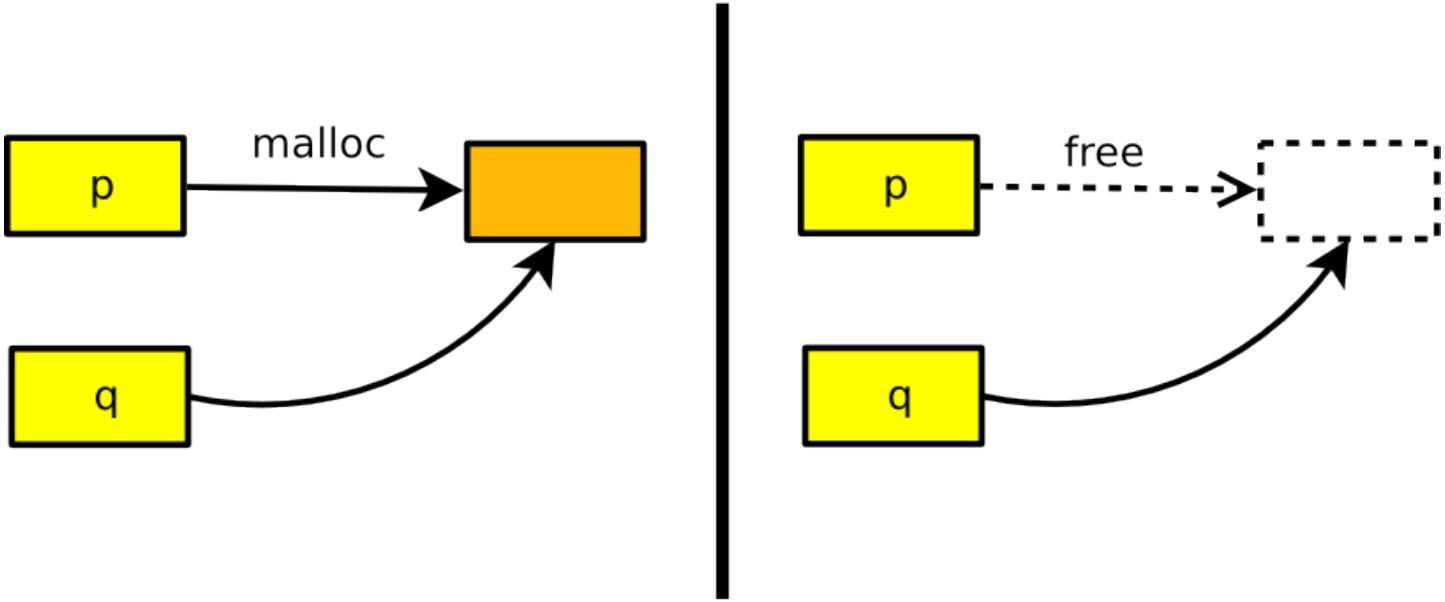
```
int main(int argc, char **argv){
        int x=2;
        int *p, *q;

        p= malloc(sizeof(int));
        q=p;

        free(p);  /* q aun mantiene referencia al heap */
                  /* puede que variable de heap sea reasignada */

        *q = x;

        return 0;
}
```

Ejemplo

Memory Leak

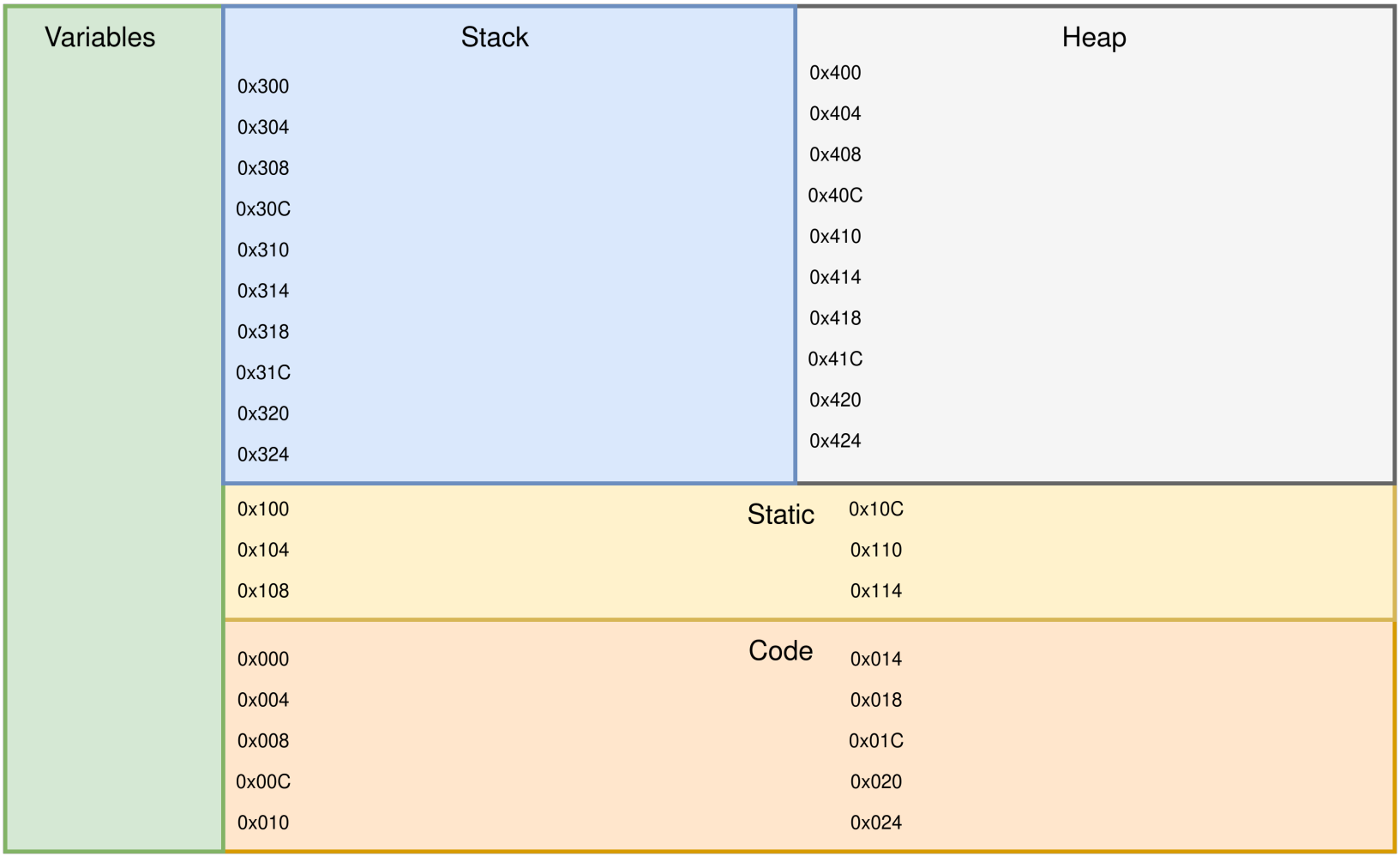


Dangling



Memoria dinámica: Ejemplo

C

```
#include<stdio.h>
#include<stdlib.h>

int main(int argc, char **argv){
        int *arreglo=NULL, i;

        arreglo = malloc(10 * sizeof *arreglo); // sizeof(int) o sizeof(*arreglo)
        // sizeof es un operador: sizeof(tipo_de_dato) o sizeof expresion
        // cast no necesario, coerción. solo para compatibilidad con C++.

        if(arreglo==NULL){
                printf("Error, memoria mal reservada...\n");
                return -1;
        }

        for(i=0;i<5;i++){
                arreglo[i]=i+1;
        }

        arreglo = realloc(arreglo, 5* sizeof(int)); //reducimos a 5 ints

        for(i=0;i<5;i++){
                printf("%d ", arreglo[i]);
        }
        free(arreglo); //libera memoria
        return 0;
}
```

Memoria dinámica: Ejemplo